Bancos de Dados Não-Relacionais

Aula 05: Redis na Prática: Estratégias de Cache (TTL), Controle de Sessão e Filas

A presentation by
Prof. Me. Reginaldo Fernandes @ Instituto Federal do Ceará – Campus Tauá Tecnologia em Análise e Desenvolvimento de Sistemas

20 de Outubro de 2026

INSTITUTO FEDERAL
Ceará
Campus Tauá

Agenda do Encontro

Projeto FastCommerce e a Persistência Poliglota

Padrão Cache-Aside e Estratégias de Expiração (TTL)

Controle de Sessão e Carrinho Atômico com Hashes

Filas Assíncronas de Checkout com Listas do Redis

Síntese e Conexão Arquitetural com o MongoDB

Projeto FastCommerce e a Persistência Poliglota

Persistência Poliglota na Arquitetura Moderna

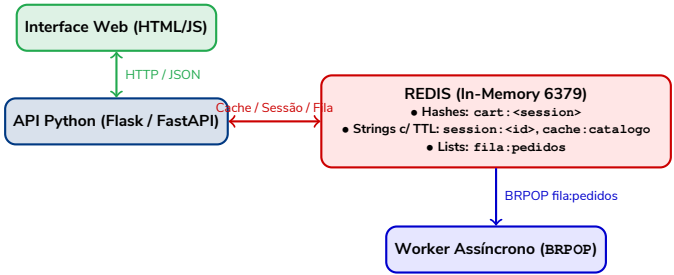
O Fim do Monolito de Dados Único

- **Cenário Tradicional:** Um único banco relacional sofrendo com leitura pesada de catálogo e filas concorrentes.
- **Persistência Poliglota:** Adoção de tecnologias NoSQL distintas especializadas em cada regra de negócio.
- **Solução em Camadas:**
  - **Redis (RAM):** Sessões rápidas, cache expirável e filas transitórias de checkout.
  - **MongoDB (Disco/BSON):** Catálogo flexível e histórico de pedidos (Aula 06).

O Projeto Transversal da Disciplina

- **FastCommerce NoSQL:** Aplicação web real desenvolvida em sala de aula.
- **Ciclo Metodológico:**
  1. **Projeto:** Modelagem de chaves e tipos no Redis.
  2. **Implementação:** Código Python com API e interface web.
  3. **Uso Prático:** Navegação, telemetria e filas.
- **Continuidade:** O mesmo código será estendido nas Aulas 06 a 08 rumo à AVP1.

Arquitetura de Dados em Tempo Real do FastCommerce



Camada de Entrada (Memória): Respostas em microssegundos (μs) para 100% das interações ativas do usuário.

Desacoplamento Assíncrono: O checkout enfileira o pedido instantaneamente, liberando a interface do usuário.

Padrão Cache-Aside e Estratégias de Expiração (TTL)

O Padrão Cache-Aside (Lazy Loading)

Mecânica de Funcionamento

- **A Aplicação Coordena:** O back-end busca na memória rápida (Redis) e na base persistente (Disco/SQL).
- **Fluxo de Leitura:**
  1. **Cache Hit:** Se a chave existir, retorna da RAM (≈ 0.4 ms).
  2. **Cache Miss:** Se não existir, busca na base lenta (≈ 120 ms), salva no Redis com `SETEX` e retorna.
- **Vantagem:** Apenas dados ativamente consultados consomem memória RAM.

Por que o Cache DEVE Ter TTL?

- **Finitude de RAM:** Evita que o Redis atinja `maxmemory` e entre em evicção forçada.
- **Autocura:** Dados defasados (*Stale Reads*) são eliminados após o tempo de vida.
- **Invalidação Pró-Ativa:** Ao atualizar o catálogo, deleta-se a chave (`DEL cache:catalogo`).

Implementação do Cache-Aside em Python no FastCommerce

Lógica de Negócio na API

- Utiliza `r.get(chave)` para consulta em memória.
- Grava no Redis com `r.setex(chave, 45, dados)` aplicando TTL de 45 segundos.
- Mensura a latência em tempo real para o painel de telemetria da interface web.
- Speedup comprovado: de $\approx$ 120 ms no Miss para $\approx$ 0.4 ms no Hit ($>$ 250$\times$ mais rápido!).

Código Python no FastCommerce

```python
@app.route("/api/produtos")
def get_produtos():
    inicio = time.time()
    chave = "cache:catalogo_produtos"

    # 1. Verifica Cache no Redis (RAM)
    if r.exists(chave):
        produtos = json.loads(r.get(chave))
        origem = "CACHE_HIT (RAM Redis)"
    else:
        # 2. Cache Miss: Busca lenta simulada
        time.sleep(0.12)
        produtos = CATALOGO_DISCO
        # 3. Grava no Redis com TTL de 45s
        r.setex(chave, 45, json.dumps(produtos))
        origem = "CACHE_MISS (Disco/DB)"

    latencia = (time.time() - inicio) * 1000
    return jsonify({
        "origem": origem,
        "latencia_ms": round(latencia, 2),
        "produtos": produtos
    })
```

Controle de Sessão e Carrinho Atômico com Hashes

Gestão de Sessão do Usuário em Memória

Por que Armazenar Sessões no Redis?

- **Escala Horizontal:** Com múltiplos servidores/containers, uma sessão local fica isolada.
- **Centralização em Memória:** O Redis atua como o *Session Store* único compartilhado.
- **Expiração Deslizante (*Sliding*):** A cada requisição, o TTL renova com `EXPIRE session:<token> 1800` (30 minutos de inatividade).

Hashes vs. Strings no Carrinho de Compras

- **String (JSON Serializado):** Exige ler o carrinho inteiro, desserializar e salvar tudo de volta. Risco grave de *Race Condition*.
- **Hashes do Redis:** A chave é o carrinho (`cart:<session>`) e os campos são os produtos (`prod_1`, `prod_2`).
- **Operação Atômica `HINCRBY`:** O Redis incrementa o campo diretamente na memória sem leitura prévia.

Operações Atômicas de Carrinho no FastCommerce

Comandos Canônicos no Terminal

- **Adicionar Item:** `HINCRBY cart:usr_10 prod_laptop 1`
- **Consultar Carrinho:** `HGETALL cart:usr_10`
- **Remover Item:** `HDEL cart:usr_10 prod_mouse`
- **Limpar Carrinho pós-checkout:** `DEL cart:usr_10`
- **Carrinhos Abandonados:** `EXPIRE cart:usr_10 3600` elimina inativos após 1 hora.

Código Python no FastCommerce

```python
@app.route("/api/carrinho/adicionar",
       methods=["POST"])
def adicionar_ao_carrinho():
    dados = request.get_json()
    prod_id = dados.get("produto_id")
    qtd = int(dados.get("quantidade", 1))

    chave = f"cart:{session_id}"

    # HINCRBY: Incremento atomico no Hash
    nova_qtd = r.hincrby(chave, prod_id, qtd)

    # Define TTL de 1 hora no carrinho
    r.expire(chave, 3600)

    return jsonify({
        "status": "success",
        "produto_id": prod_id,
        "quantidade": nova_qtd
    })
```

Filas Assíncronas de Checkout com Listas do Redis

O Problema do Checkout Síncrono no E-commerce

Por que o Checkout Síncrono Falha?

- **Cadeia Lenta:** Cartão, antifraude, NF-e e e-mail levam de 2 a 6 segundos.
- **Esgotamento:** 1000 clientes simultâneos derrubam servidores por conexões abertas.
- **Solução com Fila:** A API valida o carrinho, empacota e joga na fila do Redis (≈ 0.5 ms).

O Padrão Produtor-Consumidor FIFO

- **Produtor (API Web):** Insere na esquerda com `LPUSH fila:pedidos <dados>`.
- **Consumidor (Worker):** Retira na direita com `BRPOP fila:pedidos 5`.
- **Por que `BRPOP`?**
  Não consome CPU em loop (*busy waiting*). O processo dorme no kernel até um item chegar!

Implementação do Produtor e do Consumidor

Produtor (API Web no Checkout)

```python
@app.route("/api/checkout", methods=["POST"])
def checkout():
    # 1. Le carrinho do Redis Hash
    itens = r.hgetall(f"cart:{session_id}")
    if not itens:
        return jsonify({"erro": "vazio"}), 400

    pedido = {
        "id_pedido": f"PED-{int(time.time())}",
        "session_id": session_id,
        "total": calcular_total(itens),
        "status": "ENFILEIRADO"
    }

    # 2. LPUSH: Enfileira pedido na lista
    pos = r.lpush("fila:pedidos",
            json.dumps(pedido))
    # 3. Limpa o carrinho
    r.delete(f"cart:{session_id}")
    return jsonify({"pedido": pedido, "pos": pos})
```

Consumidor (`worker_pedidos.py`)

```python
def processar_pedidos():
    chave = "fila:pedidos"
    print("Worker escutando via BRPOP...")

    while True:
        # BRPOP bloqueia ate 5s sem CPU
        res = r.brpop(chave, timeout=5)
        if res is None:
            continue

        _, pedido_raw = res
        pedido = json.loads(pedido_raw)

        print(f"Processando:
              {pedido['id_pedido']}")
        executar_pagamento(pedido)

        # PREVIEW DA AULA 06 (MONGODB):
        # db.pedidos.insert_one(pedido)
```

Síntese e Conexão Arquitetural com o MongoDB

Checklist de Competências Consolidadas

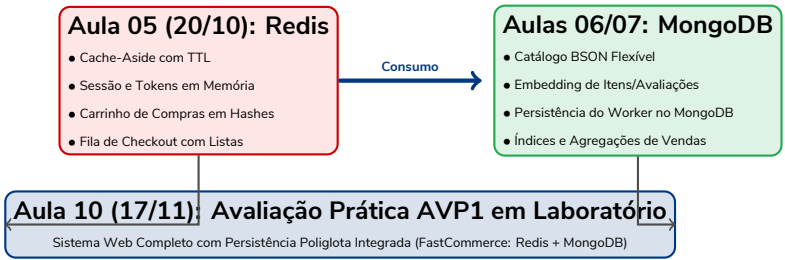
O Que Aprendemos e Executamos

- **Padrão Cache-Aside:** Implementação com expiração automática (TTL) e mensuração do Speedup de acesso em memória ($> 250\times$).
- **Gerenciamento de Sessão:** Tokens efêmeros centralizados com renovação dinâmica (*Sliding Expiration*).
- **Hashes do Redis:** Carrinho de compras atômico com `HINCRBY`, evitando condições de corrida em e-commerce.
- **Filas com Listas:** Desacoplamento assíncrono entre a API de checkout (`LPUSH`) e o worker de processamento (`BRPOP`).

Ponte com a Aula 06: O Modelo de Documentos

- **Onde o Redis Termina?**
  O Redis é excelente para dados em trânsito e alta frequência. Porém, memória RAM é cara: não guardamos histórico fiscal de 10 anos no Redis!
- **O Papel do MongoDB (Aula 06):**
  1. O catálogo de produtos passará a ser uma coleção flexível de documentos BSON (`produtos`).
  2. O worker que consome a fila do Redis salvará os pedidos confirmados na coleção `pedidos` no MongoDB.
- **Resultado:** Uma arquitetura poliglota completa e pronta para a AVP1!

Evolução do Projeto FastCommerce rumo à AVP1



Roteiro Prático: Execute o servidor web e o worker para testar o ciclo completo em laboratório!

Dúvidas & Prática em Laboratório

IFCE Campus Tauá – ADS16 (Persistência Poliglota)

FastCommerce NoSQL: Redis na Memória → MongoDB no Disco